

Swift / ...

Protocol

Hashable

A type that can be hashed into a `Hasher` to produce an integer hash value.

iOS 8.0+ | iPadOS 8.0+ | Mac Catalyst 13.0+ | macOS 10.10+ | tvOS 9.0+ | visionOS 1.0+ |
watchOS 2.0+

```
protocol Hashable : Equatable, ~Copyable, ~Escapable
```

Mentioned in

[Adopting Common Protocols](#)

Overview

You can use any type that conforms to the `Hashable` protocol in a set or as a dictionary key. Many types in the standard library conform to `Hashable`: `Strings`, integers, floating-point and Boolean values, and even sets are hashable by default. Some other types, such as optionals, arrays and ranges automatically become hashable when their type arguments implement the same.

Your own custom types can be hashable as well. When you define an enumeration without associated values, it gains `Hashable` conformance automatically, and you can add `Hashable` conformance to your other custom types by implementing the `hash(into:)` method. For structs whose stored properties are all `Hashable`, and for enum types that have all-`Hashable` associated values, the compiler is able to provide an implementation of `hash(into:)` automatically.

Hashing a value means feeding its essential components into a hash function, represented by the `Hasher` type. Essential components are those that contribute to the type's implementation of `Equatable`. Two instances that are equal must feed the same values to `Hasher` in `hash(into:)`, in the same order.

Conforming to the Hashable Protocol

To use your own custom type in a set or as the key type of a dictionary, add `Hashable` conformance to your type. The `Hashable` protocol inherits from the `Equatable` protocol, so you must also satisfy that protocol's requirements.

The compiler automatically synthesizes your custom type's `Hashable` and requirements when you declare `Hashable` conformance in the type's original declaration and your type meets these criteria:

- For a `struct`, all its stored properties must conform to `Hashable`.
- For an `enum`, all its associated values must conform to `Hashable`. (An `enum` without associated values has `Hashable` conformance even without the declaration.)

To customize your type's `Hashable` conformance, to adopt `Hashable` in a type that doesn't meet the criteria listed above, or to extend an existing type to conform to `Hashable`, implement the `hash(into:)` method in your custom type.